

# Qline configuration reference

## meta\_config.json and sim\_config.json

`gen_config` turns one `meta_config.json` into the configuration of every program of a Qline pair. For the simulator it also reads `sim_config.json`. It writes into the current directory:

| File                                                            | Program                                                                     |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>alice/gc.json</code> , <code>bob/gc.json</code>           | gc (global counter)                                                         |
| <code>alice/qber.json</code> , <code>bob/qber.json</code>       | qber                                                                        |
| <code>alice/node.json</code> , <code>bob/node.json</code>       | node (postprocessing)                                                       |
| <code>alice/kms.json</code> , <code>bob/kms.json</code>         | km-server (KMS, ETSI GS QKD 014 API)                                        |
| <code>alice/network.json</code> , <code>bob/network.json</code> | the Python hardware tooling in <code>remote/</code> and <code>local/</code> |
| <code>alice/sim.json</code> , <code>bob/sim.json</code>         | hw_sim (simulator only)                                                     |

```
gen_config -c meta_config.json           # real hardware
gen_config -c meta_config.json -s sim_config.json # simulator
gen_config                               # simulator, built-in defaults
gen_config -c meta_config.json -g       # also generate the mTLS certificates
```

**Simulator mode** (`-s`, or no arguments) differs from hardware mode in three ways:

- Alice and Bob run on one machine, so some paths get a `_alice` or `_bob` suffix to keep them apart. The tables below mark these fields with *suffixed*.
- The node keys at `node.key_path` are generated with `openssl` when they are missing, and `kms.alice_peer_id` and `kms.bob_peer_id` are derived from them. The values in the meta config are ignored.
- gc ignores the gcr timeout.

## meta\_config.json

### ip

| Field                  | Sim value              | Meaning                                                                                                                                                                                                                                                                                               |
|------------------------|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>alice</code>     | <code>127.0.0.1</code> | Alice's address on the client-side network. Alice's KMS serves its ETSI 014 API here ( <code>api_addr</code> in <code>kms.json</code> ), the <code>local/etsi14</code> scripts connect here, and it is a SAN of Alice's KMS certificate.                                                              |
| <code>bob</code>       | <code>127.0.0.1</code> | The same for Bob.                                                                                                                                                                                                                                                                                     |
| <code>alice_wrs</code> | <code>127.0.0.1</code> | Alice's address on the Alice–Bob link through the White Rabbit switch. Alice's node listens here on <code>port.node_alice</code> and is the libp2p boot node Bob dials. The <code>bind_address</code> in <code>kms.json</code> is this address's /24 network (last octet 0); the KMS does not use it. |
| <code>bob_wrs</code>   | <code>127.0.0.1</code> | Bob's address on the link. gc and qber on both sides connect to Bob here ( <code>port.gc</code> , <code>port.qber</code> ), and Bob's node announces <code>bob_wrs:port.node_bob</code> .                                                                                                             |

## port

| Field                   | Sim value | Used by                                                                                                                |
|-------------------------|-----------|------------------------------------------------------------------------------------------------------------------------|
| <code>hw</code>         | 13000     | <code>remote/hw_*.py</code> , the hardware interface server. Hardware only.                                            |
| <code>hws</code>        | 13001     | <code>remote/hws_*.py</code> , the hardware server that runs calibration and coordinates Alice and Bob. Hardware only. |
| <code>mon</code>        | 13002     | <code>remote/mon_*.py</code> , the monitoring server. Hardware only.                                                   |
| <code>kms_alice</code>  | 13003     | Alice's ETSI GS QKD 014 API.                                                                                           |
| <code>kms_bob</code>    | 13004     | Bob's ETSI GS QKD 014 API.                                                                                             |
| <code>gc</code>         | 13005     | gc: Bob listens on <code>bob_wrs:gc</code> , Alice connects.                                                           |
| <code>qber</code>       | 13006     | qber: Bob listens on <code>bob_wrs:qber</code> , Alice connects.                                                       |
| <code>node_alice</code> | 13007     | Alice's node, libp2p over TCP on <code>alice_wrs</code> .                                                              |
| <code>node_bob</code>   | 13008     | Bob's node, libp2p over TCP on <code>bob_wrs</code> .                                                                  |
| <code>showlogs</code>   | 13009     | <code>monitoring/server.py</code> , the log viewer. Hardware only.                                                     |
| <code>restartd</code>   | 13010     | <code>remote/restartd.py</code> , restarts qline services over TCP. Hardware only.                                     |
| <code>logd</code>       | 13011     | <code>remote/logd.py</code> , read-only access to the log files. Hardware only.                                        |

The hardware-only ports are only written to `network.json`.

## file

Paths of the FIFOs, sockets and files the programs exchange data through.

| Field                  | Sim value                       | Suffixed | Meaning                                                                                        |
|------------------------|---------------------------------|----------|------------------------------------------------------------------------------------------------|
| <code>gcr</code>       | <code>/tmp/gcr.fifo</code>      | no       | FIFO carrying the global counter and detection result from Bob's FPGA to gc.                   |
| <code>gc</code>        | <code>/tmp/gc.fifo</code>       | yes      | FIFO carrying the global counter from gc back to the FPGA.                                     |
| <code>angle</code>     | <code>/tmp/angle.fifo</code>    | yes      | FIFO carrying the qubit angles from the FPGA; the node and qber read it.                       |
| <code>result</code>    | <code>/tmp/result.fifo</code>   | no       | FIFO carrying Bob's click results from gc to the node and qber.                                |
| <code>gcuser</code>    | <code>" "</code>                | no       | Optional FIFO receiving a copy of the global counter for a user program. Empty disables it.    |
| <code>fpgareg</code>   | <code>/tmp/fpgareg</code>       | yes      | FPGA control registers (memory mapped). In the simulator, <code>hw_sim</code> 's command path. |
| <code>startstop</code> | <code>/tmp/gc_startstop</code>  | yes      | Unix socket for start and stop commands to gc; the node and qber send through it.              |
| <code>hw_params</code> | <code>/tmp/hw_params.txt</code> | yes      | Current hardware parameters, read by gc and <code>hw_sim</code> .                              |
| <code>kms</code>       | <code>/tmp/kms.fifo</code>      | yes      | Unix socket the node delivers final keys to its KMS through.                                   |

## kms

| Field                         | Sim value                                             | Meaning                                                                                                                                                                                                                                                                                        |
|-------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>alice_peer_id</code>    | derived                                               | libp2p peer ID of Alice's node. It names Alice's KMS ( <code>kme_config.P2P.name</code> ), which makes it the SAE ID Bob uses in ETSI requests, and it identifies the boot node. On hardware it must be the peer ID of the key at Alice's <code>node.key_path</code> .                         |
| <code>bob_peer_id</code>      | derived                                               | The same for Bob.                                                                                                                                                                                                                                                                              |
| <code>key_lifetime_ms</code>  | 3600000                                               | Time a key stays in the KMS key pool before it expires unused.                                                                                                                                                                                                                                 |
| <code>default_key_size</code> | 512                                                   | Size of the final keys in bits. It sets both the node's <code>requested_final_key_size</code> and the size the KMS delivers, so the two always match.                                                                                                                                          |
| <code>max_key_count</code>    | 100000                                                | Largest number of keys the KMS key pool holds.                                                                                                                                                                                                                                                 |
| <code>authentication</code>   | <code>false</code>                                    | Mutual TLS on the ETSI API ( <code>SAEs.mtls</code> in <code>kms.json</code> ). With <code>true</code> the API serves HTTPS and accepts only clients presenting a certificate from the configured CA with the CN <code>sae_id</code> . The certificates come from <code>gen_config -g</code> . |
| <code>ca_path</code>          | <code>./authentication/certs/ca.crt</code>            | Suffixed. CA certificate the KMS verifies client certificates against.                                                                                                                                                                                                                         |
| <code>cert_path</code>        | <code>./authentication/certs/cert.pem</code>          | Suffixed. The KMS's server certificate.                                                                                                                                                                                                                                                        |
| <code>key_path</code>         | <code>./authentication/private/eckey_pkcs8.pem</code> | Suffixed. The KMS's private key, PKCS#8.                                                                                                                                                                                                                                                       |

The three paths are written to `kms.json` as they are, so a relative path is resolved against the KMS's working directory. `gen_config -g` copies the generated files to `alice/` and `bob/` under the basenames of these paths.

## node

| Field                         | Sim value                  | Meaning                                                                                                                                                  |
|-------------------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>key_path</code>         | <code>keys/node.pk8</code> | Suffixed. The node's libp2p identity, an RSA key in PKCS#8 DER. The peer IDs are derived from it.                                                        |
| <code>qtol</code>             | 0.09                       | QBER tolerance: the highest estimated QBER at which a round is still postprocessed.                                                                      |
| <code>clicks_per_round</code> | 10000000                   | Detection windows read from the hardware per postprocessing round, two per angle byte. Larger rounds take longer and give better finite-size statistics. |
| <code>key_basis_mode</code>   | Asymmetrical, 0.9          | Which bases become key, see below.                                                                                                                       |
| <code>decoystates</code>      | set                        | Decoy-state parameters, see below. Absent disables the decoy-state analysis.                                                                             |

`key_basis_mode` is one of:

- "Symmetrical", the default: both bases go into the key, and parameter estimation sacrifices a random sample of the sifted key.
- {"Asymmetrical": {"expected\_key\_basis\_probability": p}}: only the Z basis becomes key and the X basis is published in full for estimation. p is the probability with which the source is expected to prepare Z. The node only checks the observed split against it and warns about a round that deviates. p is optional. It must equal source\_key\_basis\_probability in sim\_config.json.

#### decoystates:

| Field | Sim value | Meaning                                                                                 |
|-------|-----------|-----------------------------------------------------------------------------------------|
| mu1   | 0.23      | Mean photon number of the signal pulses.                                                |
| mu2   | 0.076     | Mean photon number of the decoy pulses, lower than mu1.                                 |
| p1    | 0.55      | Probability of a pulse being signal; decoy is 1 - p1.                                   |
| esec  | 1e-10     | Secrecy failure probability in the final key length bound.                              |
| ecor  | 1e-10     | Correctness failure probability in the final key length bound.                          |
| K     | 19        | Number of security parameters the secrecy budget is split over (Rusca one-decoy bound). |

mu1, mu2 and p1 must equal decoy\_states in sim\_config.json.

## tls

Only used by gen\_config -g. The block is optional; its defaults are the values shown.

| Field                                   | Default                                    | Meaning                                                                                  |
|-----------------------------------------|--------------------------------------------|------------------------------------------------------------------------------------------|
| country, state, locality, org, org_unit | FR, Paris, Paris, VeriQloud, Veriqloud-KME | Subject fields of the certificates.                                                      |
| ca_cn                                   | VQKME                                      | Common name of the root CA.                                                              |
| validity_days                           | 1095                                       | Validity of every generated certificate.                                                 |
| ec_curve                                | secp384r1                                  | Elliptic curve of every generated key.                                                   |
| extra_sans                              | []                                         | SANs added to both KMS server certificates, e.g. "DNS:kms.example.com" or "IP:10.0.0.5". |

-g creates one EC root CA in certs/. The CA signs a server certificate for each KMS and one client certificate with the CN sae\_id. Each server certificate carries the SANs IP:<ip.alice or ip.bob>, DNS:localhost and IP:127.0.0.1, plus extra\_sans. The client bundle (ca.crt, sae\_cert.pem, sae\_key.pem) is written to client/; the local/etsi14 scripts read it with --auth from \$KMS\_CLIENT\_CERTS. Running -g again replaces the whole chain.

## Fixed values

These values in the generated files do not come from the meta config:

| Value         | Where                                                            |
|---------------|------------------------------------------------------------------|
| SAE ID sae_id | the single SAE in kms.json, and the CN of the client certificate |

| Value                                                                                                                          | Where                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| KMS IDs <code>alice_kme</code> , <code>bob_kme</code>                                                                          | <code>kme_id</code> in <code>kms.json</code>                                |
| <code>max_key_per_request</code> 100, <code>min_key_size</code> 128, <code>max_key_size</code> 4096                            | <code>kms.json</code> ; the KMS delivers <code>default_key_size</code> only |
| <code>static_angles</code> [0, 32, 96, 64]                                                                                     | <code>node.json</code>                                                      |
| ready flag <code>/tmp/qkd_ready</code> ; idle flag <code>/tmp/node_idle</code> (sim: <code>_alice</code> , <code>_bob</code> ) | <code>gc.json</code>                                                        |
| log level <code>Info</code>                                                                                                    | all programs                                                                |

## sim\_config.json

Both simulators read the same values; only their FIFO paths differ.

| Field                               | Shipped         | Meaning                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>angles</code>                 | [0, 32, 96, 64] | Angle codes the source prepares, as the hardware writes them: 0 and 64 are the Z basis, 32 and 96 the X basis. Keep as shipped.                                                                                                                                                                                                                                                           |
| <code>seed</code>                   | 42              | Seed of the random stream both simulators share. The stream restarts at every session start, so two runs see identical photon data.                                                                                                                                                                                                                                                       |
| <code>eta</code>                    | 0.01            | Single-photon transmission of the channel.                                                                                                                                                                                                                                                                                                                                                |
| <code>qberr</code>                  | 0.02            | Optical error rate: a number, or <code>{"type": "Fixed", "value": v}</code> , <code>{"type": "Uniform", "min": a, "max": b}</code> or <code>{"type": "Gaussian", "mean": m, "std_dev": s}</code> , redrawn for every batch.                                                                                                                                                               |
| <code>pulse_distance</code>         | 12.5e-9         | Time between pulses in seconds; 12.5 ns is 80 MHz.                                                                                                                                                                                                                                                                                                                                        |
| <code>dead_time</code>              | 15e-6           | Detector dead time in seconds: no click follows another within it, which caps the count rate at $1/\text{dead\_time}$ . 0 disables it.                                                                                                                                                                                                                                                    |
| <code>dark_count_probability</code> | 1.25e-6         | Dark-count probability per gate. A dark count rate D in counts per second is $D \cdot \text{pulse\_distance}$ ; 1.25e-6 is 100 cps at 12.5 ns. 0 disables it.                                                                                                                                                                                                                             |
| <code>afterpulse</code>             | two components  | Afterpulse model, at most two <code>{tau, p_ap}</code> components: <code>tau</code> is the detrapping time constant in seconds, <code>p_ap</code> the afterpulses per avalanche at zero hold-off, which can exceed 1. The dead time reduces it to $p\_ap \cdot e^{(-\text{dead\_time}/\text{tau})}$ . The shipped values are the measured AUREA InGaAs SPAD. Empty disables afterpulsing. |
| <code>software_filter</code>        | 0.25            | Fraction of the hardware gate the software gate keeps. Photon clicks are all kept; dark counts and afterpulses only with this probability. 1.0 disables filtering.                                                                                                                                                                                                                        |
| <code>speedup</code>                | 10.0            | Speed relative to real time. 10 delivers ten seconds of hardware output per second; the data is identical to a real-time run with the same seed.                                                                                                                                                                                                                                          |

| Field                                     | Shipped                                                                   | Meaning                                                                                                                                                                                                                       |
|-------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>source_key_basis_probability</code> | 0.9                                                                       | Probability of the source preparing the Z basis, for the asymmetric protocol; the detector is biased to $1 - p$ . Absent draws all four angles uniformly. Must equal the node's <code>expected_key_basis_probability</code> . |
| <code>decoy_states</code>                 | <code>mu1</code> 0.23,<br><code>mu2</code> 0.076,<br><code>p1</code> 0.55 | Source intensities. Absent makes the source emit single photons. Must equal the node's <code>decoystates</code> .                                                                                                             |

## Checks

`gen_config` compares the two files where they describe the same thing:

- `decoy_states` and `node.decoystates` with different `mu1`, `mu2` or `p1`: error. Set on one side only: warning.
- `source_key_basis_probability` and `expected_key_basis_probability` differing: error. Set on one side only: warning.

## Docker image

The simulator stack image `ghcr.io/blindqlouder/qline-stack` runs `gen_config -c meta_config.json -s sim_config.json` at every start on a directory mounted at `/config`, and writes `alice/`, `bob/` and `keys/` into it. On top of `gen_config`:

- The derived peer IDs are written back into `kms.alice_peer_id` and `kms.bob_peer_id`, so the directory works as `QLINE_CONFIG_DIR` for `local/etsi14`.
- The node keys persist in `keys/`, so the peer IDs stay the same across restarts.
- With `kms.authentication` `true`, the first start runs `-g`, and `kms.json` is pointed at the certificates in `alice/` and `bob/`. The chain is kept until `client/` is deleted.
- `ip.*` must stay `127.0.0.1`, since the whole pair runs in one container. Publish `port.kms_alice` and `port.kms_bob` with `-p`.